

DSC 670 Milestone 2

Commercial Fleet EV Incentive Assistant

Dan Morrow

DSC 670: Generative AI

This notebook contains both the 'formal project documentation', prompt experiments, code, model responses, and evaluation in one place. The goal is to show not only what was tested, but also the generative AI concepts behind the design and what the experiment results mean for the project.

Project Introduction

EV charging projects can become harder to move forward once the conversation turns to cost. Commercial fleet customers may be interested in electrification but still have limited visibility into programs that can reduce the cost of charging equipment, installation, or related electrical infrastructure.

Those cost saving opportunities may come from electric utilities as well as federal, state, and local governments. The challenge is that the information is spread across different sources and can change at any point. Eligibility rules, available funding, deadlines, and program requirements are not always easy to interpret during a sales conversation.

This project proposes a generative AI assistant that gives a sales representative a faster way to identify potentially relevant programs, understand their requirements, explain possible savings, and recognize when something still needs to be verified before it is presented to a customer.

Updated Problem Statement

Milestone 1 treated state, local, and utility incentives as parts of one broad search. Feedback was taken from my leader and applied; now the revised project keeps those sources but gives the workflow a clearer order.

The assistant will begin with the customer's electric utility because utility-sponsored programs are directly connected to the customer's service territory and may include charging-equipment rebates, installation support, make-ready assistance, time-of-use rates, managed charging, or other programs related to EV charging. The Alternative Fuels Data Center (AFDC) specifically includes utility incentives for nonresidential

customers, including commercial and fleet users (U.S. Department of Energy, Alternative Fuels Data Center [AFDC], n.d.-c).

After the utility search, the assistant will expand to applicable federal, state, and local programs. Government programs remain important, but they can add jurisdictional rules, application steps, funding limits, and documentation requirements.

The goal is not to guarantee eligibility. Instead, the assistant should identify programs that appear relevant, explain why they may apply, estimate possible savings only when the published rules support a calculation, and clearly identify what still needs to be verified with the utility or program administrator.

This approach should help simplify the workflow. Instead of trying to prove that every possible incentive has been found, the assistant should show which information it used, which programs appear relevant, and where uncertainty remains, so a human can confirm the accuracy before giving it to a customer.

Generative AI Approach

The generative AI component for this milestone is a text-based large language model (LLM). The five prompt experiments in this milestone use Google Gemini 3.5 Flash-Lite through the Gemini API. The same model is applied to every experiment, this way the results are driven mainly by the prompt and the information supplied to the model rather than by switching models between tests.

An LLM does not work like a traditional database that looks up one stored answer. Text is broken into smaller units called tokens, and the model uses patterns learned during training along with the context in the current prompt to predict a sequence of likely next tokens. That process allows the model to generate new text, summarize information, compare conditions, and respond to natural-language questions.

That flexibility is needed for this project because a salesperson needs more than just a list of incentive names. The assistant needs to interpret program language, explain requirements in plain language, ask follow-up questions, compare conditions, and organize the result into something useful that the sales person can use in a conversation with the customer.

However, the same flexibility also has potential risks. A language model can generate a confident answer even when the supplied information is incomplete or ambiguous. That is especially important here because an incorrect assumption about a utility, deadline, eligible cost, or charging-port count could materially change the incentive being discussed. This could erode the brand of the sales person and the customers trust in that sales person. The model needs to have clear boundaries around what it can infer and when it should stop and ask for more information.

Retrieval, Prompt Engineering, and Fine-Tuning

This project utilizes three concepts that play different roles in the proposed assistant: retrieval, prompt engineering, and fine-tuning.

Retrieval is responsible for bringing current incentive information into the conversation. Program amounts, deadlines, available funding, and eligibility requirements can change, so those facts should come from current source material rather than relying on what the language model may have learned during training.

Prompt engineering controls how the model is asked to behave at the time a request is made. A prompt can define the model's role, provide customer and program context, tell the model what it should not assume, and specify how the response should be organized. The experiments in this milestone test whether those instructions are enough to produce the behavior needed for the assistant.

Fine-tuning is different because it uses additional training examples to shape model behavior rather than putting all of the guidance into each individual prompt. A curated set of examples could teach patterns such as stopping when a required input is missing, separating confirmed facts from assumptions, and refusing to produce an unsupported savings estimate.

For this project, retrieval should keep the program facts current, prompting should provide immediate control over the response, and fine-tuning will help with behaviors that remain inconsistent after the prompts are improved. In other words, retrieval keeps the information current while prompting and fine-tuning control how the model behaves with that information.

Data Sources and Project Feasibility

Alternative Fuels Data Center

AFDC is the strongest primary source for the initial prototype because its laws and incentives data can be utilized through a developer API. Available fields include state, title, descriptive program text, dates, technology categories, incentive categories, and other classifications that can be used to narrow records to EV charging and commercial fleet use cases (AFDC, n.d.-b).

AFDC also includes utility and private-organization incentives, which supports the revised utility-first workflow. However, AFDC notes that not every service or incentive offered by utilities and private organizations is included because of the number of programs available (AFDC, n.d.-a). The assistant should therefore treat AFDC as a strong starting point rather than claim it contains every possible utility opportunity.

Utility and Location Data

The customer's utility provider should be entered directly whenever the salesperson already knows it. When the provider is unknown, location data can help narrow the possibilities.

The U.S. Census Bureau Geocoder can convert a street address into geographic information that can help establish the correct state, county, and local jurisdiction (U.S. Census Bureau, n.d.). The U.S. Energy Information Administration's Form EIA-861 service-territory data can then help identify utilities that operate in that area (U.S. Energy Information Administration [EIA], n.d.).

That process still does not prove which utility serves a specific street address. Multiple utilities can operate in the same county, so the final assistant should require the salesperson to confirm the utility before utility-specific programs are treated as applicable.

Supplemental Sources and Verification

The Database of State Incentives for Renewables and Efficiency (DSIRE) can provide supplemental coverage and supports program searches by geography, sector, utility, incentive type, and technology (N.C. Clean Energy Technology Center, n.d.). Because API access may require a key or subscription, the initial prototype should not depend on DSIRE being available.

Utility websites and original program-administrator pages will remain important verification sources. Structured data can help discover potentially relevant programs, but the original source is better suited for confirming current eligibility language, deadlines, funding status, and application requirements.

Initial System Design

A salesperson will begin by entering the customer and project information that is already known, such as the business location, organization type, planned charging needs, expected installation timing, and utility provider when available. The assistant should not require every project detail up front because some information, such as the final charger configuration or project cost, may not be known early in the sales process.

The first search will focus on utility-sponsored opportunities associated with the confirmed provider. The second search will expand to relevant federal, state, and local programs. Retrieved program information will then be passed to the language model along with the customer information.

The model's role is to explain the program requirements, identify why a program may be relevant, and determine what additional information is needed to evaluate it more closely. If a specific incentive depends on project cost, charging-port count, or another detail that has not been confirmed yet, the assistant should ask for that information rather than making an assumption.

When a program includes a clear mathematical rule and the required inputs are available, the Milestone 2 prototype uses a separate Python function to perform the calculation. This keeps the arithmetic transparent and easier to verify instead of relying on the language model to calculate a financial result simply because it can produce a plausible-looking number.

That separate function works well as a proof of concept, but it may not be the final production architecture. A larger system may eventually need a more scalable rules or calculation layer that can support many different incentive structures without creating a custom function for every program.

Import the Required Packages

The notebook uses the Google Gemini API for the prompt experiments. `getpass` is used to enter the API key without storing it directly in the notebook, `json` is used to format the customer and program records inside the prompts, and the IPython display tools are used to show model responses in the notebook.

Set Up the API

The API key is entered securely and used to create the Gemini client. The same Gemini model will be used across the five experiments so the responses can be compared under consistent conditions.

The code and stored output below are left exactly as they were executed in the original notebook.

```
In [4]: from google import genai
        from getpass import getpass

        api_key = getpass("Enter your Gemini API key: ")

        client = genai.Client(api_key=api_key)
```

```
In [10]: #Import packages
         import json
         #Figured out 10 cells later I needed this
         from google import genai
```

```
from getpass import getpass
from IPython.display import Markdown, display
```

Sample Fleet Customer Profile

The same fictional fleet customer is used throughout the prompt experiments so the model is working from a consistent starting point. The customer represents a commercial fleet preparing for a Level 2 charging project.

The sample profile is:

- Location: Omaha, Nebraska
- Organization type: Commercial fleet
- Utility provider: Example Electric Utility
- Planned Level 2 chargers: 10
- Estimated project cost: \$150,000
- Expected installation year: 2027

These values are intentionally simple so the experiments can focus on how the model handles incentive information rather than on a complicated customer scenario. Some experiments intentionally remove or question individual pieces of the profile to see whether the model recognizes missing information, asks for clarification, or makes assumptions.

Project Cost was added here for testing purposes only, at this point, we do not know the overall project cost but it does help the testing later.

```
In [5]: #Create the sample customer
customer = {
    "city": "Omaha",
    "state": "Nebraska",
    "organization_type": "Commercial fleet",
    "utility_provider": "Example Electric Utility",
    "planned_level2_chargers": 10,
    "estimated_project_cost": 150000,
    "expected_installation_year": 2027
}

display(customer)
```

```
{'city': 'Omaha',
 'state': 'Nebraska',
 'organization_type': 'Commercial fleet',
 'utility_provider': 'Example Electric Utility',
 'planned_level2_chargers': 10,
 'estimated_project_cost': 150000,
 'expected_installation_year': 2027}
```

Create Sample Incentive Records

The first round of prompt testing uses small illustrative program records rather than connecting immediately to a live incentive database. This keeps the test controlled: when the model responds, it is easier to tell whether it is using the supplied information correctly or introducing something that was never provided.

The utility example includes a rebate per Level 2 charging port, a percentage-of-cost limit, a maximum rebate, a prior-approval requirement, and funding availability language. The government example includes a percentage award, a maximum award, a location requirement, an application-timing requirement, and a grant-period requirement.

These sample records are not intended to represent current real-world programs. A later phase of the project can replace them with verified AFDC, utility, and government program information once the model behavior is better understood.

```
In [6]: #Create a sample utility program
utility_program = {
    "program_name": "Commercial EV Charging Rebate",
    "program_type": "Utility",
    "eligible_customer": "Nonresidential electric customers",
    "program_text": (
        "Eligible nonresidential customers may receive $2,000 per Level 2 ch
        up to 50% of eligible equipment and installation costs, with a maxi
        Customers must receive program approval before purchasing equipment
        Funding is subject to availability."
    ),
    "source": "Illustrative record for prompt testing"
}

#Create a sample government program
government_program = {
    "program_name": "Commercial Charging Infrastructure Grant",
    "program_type": "State",
    "eligible_customer": "Businesses operating in the state",
    "program_text": (
        "Eligible businesses may receive up to 40% of approved charging infr
        with a maximum award of $50,000. Applicants must operate in the sta
        before construction begins, and complete the project during the app
    ),
    "source": "Illustrative record for prompt testing"
}

display(utility_program)
display(government_program)

{'program_name': 'Commercial EV Charging Rebate',
 'program_type': 'Utility',
 'eligible_customer': 'Nonresidential electric customers',
 'program_text': 'Eligible nonresidential customers may receive $2,000 per L
 evel 2 charging port, up to 50% of eligible equipment and installation cost
 s, with a maximum rebate of $20,000. Customers must receive program approval
 before purchasing equipment. Funding is subject to availability.',
 'source': 'Illustrative record for prompt testing'}
```

```
{'program_name': 'Commercial Charging Infrastructure Grant',
  'program_type': 'State',
  'eligible_customer': 'Businesses operating in the state',
  'program_text': 'Eligible businesses may receive up to 40% of approved charging infrastructure costs, with a maximum award of $50,000. Applicants must operate in the state, submit an application before construction begins, and complete the project during the approved grant period.',
  'source': 'Illustrative record for prompt testing'}
```

Build a Reusable API Function

The same API function will be used for all five prompt experiments so the model is being tested under the same basic conditions each time. Keeping the model and API call consistent makes the comparisons cleaner because the main things changing from one experiment to the next are the prompt and the information supplied to the model.

This also helps separate prompt behavior from technical differences in how a request is sent. If one experiment performs better or worse, the result should reflect how the model handled that scenario rather than a different API setup.

```
In [13]: #Import display tools
from IPython.display import Markdown, display

#Select the Gemini model
model_name = "gemini-3.5-flash-lite"

#Create a reusable API function
def run_prompt(prompt):
    response = client.models.generate_content(
        model=model_name,
        contents=prompt
    )
    return response.text

display(Markdown("**Output:** Reusable prompt function created."))
```

Output: Reusable prompt function created.

Prompt Experiment Design

Prompt engineering is the first behavioral control being tested because it does not require changing the underlying model. Each prompt gives the model a role, supplies a customer and program scenario, and adds instructions about what should or should not be assumed.

The five experiments intentionally test different failure points instead of asking the same question five different ways. Together they look at missing customer information, utility-program interpretation, government-program interpretation, savings calculations, and conflicting source information.

The full prompt used for each experiment is shown in the code cell for that experiment. The model output is preserved exactly as it was originally generated so the evaluation reflects the actual test rather than a regenerated response. The controlled sample record contains more information than may be available at the initial customer intake because some experiments need different fields to test different behaviors. This single approach may change as fine-tuning is needed.

Prompt Experiment 1: Missing Customer Information

The first experiment tests whether the model recognizes when important customer or project information is missing. The utility provider is intentionally removed from the customer profile before the prompt is sent.

The utility provider matters because a utility-sponsored incentive is only useful if that utility actually serves the customer. The model should not solve around that missing fact by simply assuming the program belongs to the customer's location.

Expected behavior: The model should stop and ask for the missing utility information before making an eligibility or savings estimate.

```
In [14]: #Remove the utility provider for this experiment
customer_missing_utility = customer.copy()
customer_missing_utility.pop("utility_provider")

#Create the first prompt
prompt_1 = f'''
You are assisting a sales representative with a commercial fleet EV charging

Customer information:
{json.dumps(customer_missing_utility, indent=2)}

Program information:
{json.dumps(utility_program, indent=2)}

Determine whether there is enough information to evaluate this program for t
If important information is missing, ask only the questions needed before ma
or savings estimate. Do not assume missing facts.
'''

#Run the prompt
experiment_1 = run_prompt(prompt_1)

display(Markdown("**Output:**"))
display(Markdown(experiment_1))
```

Output:

Based on the program and customer information provided, we have enough information to make an initial eligibility and savings estimate.

Here is how the program applies to the customer:

- **Eligibility:** As a commercial fleet in Omaha, they are a nonresidential electric customer, meaning they meet the basic customer type requirement (assuming they are served by the utility offering the program).
- **Rebate Calculation:**
 - They plan to install 10 Level 2 chargers. *(Note: We would just need to confirm if "10 chargers" refers to 10 individual ports, as the rebate is calculated per port).*
 - At 2,000 *per port*, *the potential rebate for 10 ports is* * *20,000**.
 - This is well within the 50% cost cap (75,000) *and hits the maximum rebate limit of* * *20,000**.

One quick clarifying question for the customer: Do the 10 planned Level 2 chargers each have a single port, or are any of them dual-port units (meaning a total of more than 10 ports)?

Experiment 1 Evaluation

The model partially followed the prompt, but it did not handle the missing information the way I expected. The utility provider was intentionally removed from the customer information, but the model still moved forward with an initial eligibility and savings estimate.

It did acknowledge that the customer would need to be served by the utility offering the program, but it treated that as an assumption instead of stopping and asking for the utility provider first.

The model did identify another useful question by asking whether the 10 planned chargers represent 10 individual ports or whether any are dual-port units. That was a good catch because the rebate is calculated per port.

This experiment shows an important distinction between **recognizing uncertainty** and **acting appropriately on uncertainty**. The model noticed that something was not confirmed, but it still continued. For this project, a confirmed utility provider should be treated as a required input that stops the evaluation until it is known.

Note: Some dollar-sign formatting in the stored model output rendered strangely when Markdown interpreted it. I am leaving the original output unchanged so the experiment result isn't regenerated.

Prompt Experiment 2: Utility Program Interpretation

The second experiment gives the model the complete sample customer profile and a utility-sponsored program.

This test is less about finding missing information and more about whether the model can translate program language into something useful for a salesperson without turning a possible match into a promise of eligibility.

Expected behavior: The model should explain why the program may be relevant, separate confirmed information from assumptions, identify what still needs verification, and avoid stating that the customer definitely qualifies.

```
In [15]: #Create the second prompt
prompt_2 = f'''
You are assisting a sales representative with a commercial fleet EV charging

Customer information:
{json.dumps(customer, indent=2)}

Utility program:
{json.dumps(utility_program, indent=2)}

Explain in plain language whether this program appears relevant to the customer.
Separate confirmed information from assumptions.
Identify any requirements that still need to be verified.
Do not state that the customer qualifies unless the supplied information clearly indicates so.
Keep the response concise and useful for a sales conversation.
'''

#Run the prompt
experiment_2 = run_prompt(prompt_2)

display(Markdown("**Output:**"))
display(Markdown(experiment_2))
```

Output:

Here is a quick evaluation of the **Commercial EV Charging Rebate** program for your upcoming Omaha fleet project.

Relevance Summary

This utility program appears **highly relevant** to your project, as it provides financial incentives specifically for commercial Level 2 EV chargers.

Confirmed Information

- **Customer Type:** The organization is a commercial fleet and a nonresidential electric customer in Omaha, matching the utility's "nonresidential electric customers" eligibility rule.
- **Hardware Planned:** You plan to install 10 Level 2 chargers (assuming 1 port per charger, this equals 10 ports).
- **Project Scale:** The estimated cost is \$150,000 for 2027 installation.

Assumptions

- **Port Count:** The rebate pays out per *port*. We are assuming your 10 planned Level 2 chargers have 1 port each (10 ports total). If any are dual-port chargers (20 ports total), the potential rebate calculation changes.
- **Utility Match:** We are assuming "Example Electric Utility" is the actual provider servicing your specific Omaha depot location.

Requirements to Verify

1. **Prior Approval Rule:** The utility strictly requires **program approval *before* purchasing equipment**. You must submit the application before signing any equipment purchase orders.
2. **Cap Calculations:** At 2,000 *per port*, 10 *ports would equal a* 20,000 rebate. This hits the utility's hard cap of 20,000 (*and is well under the* 5075,000). We need to confirm whether your 10 chargers translate to 10 or 20 total ports.
3. **Funding Availability:** Program text notes funding is "subject to availability." We should verify if funds open on an annual cycle that will accept applications for a 2027 installation.

Experiment 2 Evaluation

The model handled this experiment fairly well. It separated the information it considered confirmed from the assumptions and requirements that still needed to be checked. It also caught two important issues: confirming that the listed utility actually serves the customer and confirming whether the 10 chargers represent 10 individual ports.

It also correctly called out the prior-approval requirement and the fact that funding is subject to availability. Those are exactly the kinds of details a salesperson needs before presenting the program as a real opportunity.

One thing I found interesting was that it described the customer as a confirmed nonresidential electric customer. The sample data identifies the organization as a commercial fleet, but it never explicitly confirms the electric account classification. That should probably have been treated as another item to verify.

Overall, this was a positive result because the model turned the program language into a useful sales explanation while still identifying several areas that needed confirmation.

Prompt Experiment 3: Government Program Interpretation

The third experiment uses a government incentive with more detailed eligibility, location, timing, and application requirements.

Government programs can be more complicated than a simple utility rebate because a salesperson may need to understand jurisdiction, application timing, eligible costs, and funding rules at the same time. The model needs to organize that information without filling in details that are not in the source.

Expected behavior: The model should organize the requirements clearly, separate confirmed requirements from assumptions, and identify what the customer still needs to confirm without promising eligibility.

```
In [16]: #Create the third prompt
prompt_3 = f'''
You are assisting a sales representative with a commercial fleet EV charging

Customer information:
{json.dumps(customer, indent=2)}

Government program:
{json.dumps(government_program, indent=2)}

Summarize this program for the sales representative.
Separate confirmed eligibility requirements from assumptions.
Identify any location, timing, or application requirements.
List any customer information that still needs to be confirmed.
Keep the explanation short and practical.
'''

#Run the prompt
experiment_3 = run_prompt(prompt_3)
```

```
display(Markdown("**Output:**"))
display(Markdown(experiment_3))
```

Output:

Program Summary: Commercial Charging Infrastructure Grant

- **Benefit:** Up to 40% of approved charging infrastructure costs, with a maximum award of \$50,000.
- **Estimated Potential Award for this Project:**
50,000(*capped at the max award, since* 150,000 project cost is \$60,000).

Confirmed Eligibility Requirements vs. Assumptions

- **Confirmed Eligibility Requirements:**
 - Must be a business operating within the state.
 - Must apply *before* construction begins.
 - Must complete the project within the approved grant period.
- **Assumptions:**
 - Assumes the customer's specific commercial fleet operations qualify as an eligible "business operating in the state" under the program guidelines.

Location, Timing, and Application Requirements

- **Location:** Must be located within the state.
- **Timing:** The application must be submitted and approved *before* any construction starts, and the project must be completed within the designated grant period.
- **Application Note:** 2027 installation is well into the future; ensure the sales rep checks if the grant will still be active or accepting applications in 2026/2027.

Customer Information Still Needed to Confirm

1. **Exact Operating Location:** Confirm they operate within the state jurisdiction required by the grant.
2. **Project Timeline:** Determine the exact planned start date for construction to ensure the grant application is submitted beforehand.
3. **Approved Cost Breakdown:** Verify which portions of the \$150,000 estimated cost qualify as "approved charging infrastructure" under the program rules.

Experiment 3 Evaluation

The model did a good job separating the program requirements from the assumptions and clearly identified the location, timing, and application rules. It also recognized that

the customer's exact operating location, construction timeline, and eligible project costs still needed to be confirmed.

There were a couple of places where it went further than the information supplied. The sample program only says the business must operate "in the state," but it never identifies which state the program applies to. Since the customer is in Nebraska, the better response would have been to ask whether this is a Nebraska program instead of treating the location requirement as matched.

The model also estimated a potential award before confirming how much of the \$150,000 project cost was actually eligible. In addition, it changed the source language from requiring an application to be submitted before construction to implying that approval had to occur before construction. That is a subtle difference, but it matters when the assistant is supposed to stay grounded in program rules.

This experiment highlights a design tradeoff. The assistant needs to give a salesperson enough information to actually do something with the result, but it cannot become so helpful that it changes the meaning of the source or converts assumptions into facts.

Add a Rebate Calculation Function

The model can help determine whether there is enough information to estimate a rebate, but I do not want it doing the actual arithmetic in this prototype. Once the program rule and required inputs are clear, Python can perform the calculation in a transparent and repeatable way.

For the sample utility program, the rebate is controlled by three limits: the amount available per charging port, the percentage of eligible project cost that can be covered, and the overall maximum rebate. The function calculates those limits and returns the lowest amount because that is how the sample rule is structured.

Separating interpretation from arithmetic also makes the result easier to check if a salesperson or customer asks how the estimate was reached. The limitation is scalability: a production system may eventually need a more flexible rules engine rather than a custom function for every type of program.

```
In [17]: #Create a rebate calculation function
def calculate_port_rebate(chargers, rebate_per_charger, project_cost, percent_limit, maximum_rebate):
    port_rebate = chargers * rebate_per_charger
    project_limit = project_cost * percent_limit
    return min(port_rebate, project_limit, maximum_rebate)

display(Markdown("**Output:** Rebate calculation function created."))
```

Output: Rebate calculation function created.

Prompt Experiment 4: Savings Calculation

The fourth experiment tests whether the model can determine when there is enough information to support a rebate estimate and whether it can correctly identify the rules that control the calculation.

This is intentionally restrictive. The model should not see a few numbers and immediately start calculating. It first needs to determine which values actually matter and whether those values have been confirmed. In this example, that includes the charging-port count, the rebate per port, the percentage limit, the eligible project cost, and the maximum rebate.

Note: The project cost is here to test a percentage based rebate.

Expected behavior: The model should identify the calculation rule and required numeric inputs, but Python should handle the arithmetic only after the required inputs are confirmed.

```
In [18]: #Create the fourth prompt
prompt_4 = f'''
You are reviewing a commercial EV charging incentive.

Customer information:
{json.dumps(customer, indent=2)}

Program information:
{json.dumps(utility_program, indent=2)}

Determine whether enough information is available to calculate an estimated
If it is, identify the exact calculation rule and list the numeric inputs ne
Do not perform the arithmetic.
If anything required is missing or ambiguous, explain what must be clarified
'''

#Run the prompt
experiment_4 = run_prompt(prompt_4)

display(Markdown("**Model Output:**"))
display(Markdown(experiment_4))

#Calculate the rebate with Python
estimated_rebate = calculate_port_rebate(
    chargers=customer["planned_level2_chargers"],
    rebate_per_charger=2000,
    project_cost=customer["estimated_project_cost"],
    percent_limit=0.50,
    maximum_rebate=20000
)

display(Markdown(f"**Python Calculation Output:** Estimated rebate = ${estim
```

Model Output:

Based on the provided customer and program information, **enough information is available** to determine the estimated rebate and its governing calculation rules.

Here are the details of the calculation:

Exact Calculation Rule

The rebate is determined by the lowest of three caps/formulas:

1. **Per-port cap:** \$2,000 per Level 2 charging port.
2. **Percentage cap:** 50% of total eligible equipment and installation costs (\$150,000).
3. **Maximum total rebate cap:** \$20,000 per project.

Numeric Inputs Needed

To apply the rules above, the following inputs (all present in the data) are used:

- **Number of Level 2 charging ports:** 10 ports (Note: The customer profile states "planned_level2_chargers: 10", assuming 1 port per charger, though the exact port count should technically be confirmed).
- **Estimated project cost:** \$150,000.

Items to Clarify (Pre-requisites for disbursement, not calculation)

While the mathematical estimate can be calculated, the program text notes a procedural requirement that must be clarified for the project to successfully receive funds:

- **Program Approval:** The customer must verify that they will receive program approval *before* purchasing the equipment (scheduled for 2027).

Python Calculation Output: Estimated rebate = \$20,000

Experiment 4 Evaluation

The model correctly identified the three limits used in the rebate calculation: the amount per charging port, the 50% project-cost limit, and the \$20,000 maximum rebate. It also identified the customer's estimated project cost and recognized that the number of charging ports was needed.

The key issue is that the customer information lists 10 chargers while the program calculates the rebate per port. The model noticed that the exact port count should technically be confirmed, but it still decided there was enough information to move forward by assuming one port per charger.

Using that assumption, the separate Python function calculated an estimated rebate of \$20,000. The arithmetic worked as expected, but the model should have confirmed the port count before passing that value to the calculation function.

This is another example of the same behavior seen in Experiment 1: the model can recognize an ambiguity and still continue anyway. The final assistant needs a stronger rule that a required but unconfirmed input stops the calculation.

Prompt Experiment 5: Conflicting Information

The final experiment tests how the model handles two sources that appear to describe the same incentive program but contain different financial terms.

Conflicting information is realistic because incentive details can change over time and different sources may not be updated at the same time. For this test, the two sources use different rebate amounts per port and different maximum rebate limits.

Expected behavior: The model should identify the conflicts, avoid choosing one source without evidence, and clearly explain what must be verified before the incentive is presented to a customer.

```
In [19]: #Create conflicting source information
source_a = {
    "source": "Source A",
    "program_text": "The program provides $2,000 per Level 2 charging port w
}

source_b = {
    "source": "Source B",
    "program_text": "The program provides $1,500 per Level 2 charging port w
}

#Create the fifth prompt
prompt_5 = f'''
You are reviewing two sources that describe the same commercial EV charging

{json.dumps(source_a, indent=2)}

{json.dumps(source_b, indent=2)}

Compare the two sources and identify every meaningful conflict.
Do not choose one source without evidence supporting that decision.
Explain what should be verified before the incentive is presented to a custo
Keep the response concise.
'''

#Run the prompt
experiment_5 = run_prompt(prompt_5)
```

```
display(Markdown("**Output:**"))
display(Markdown(experiment_5))
```

Output:

Conflict Identification

The two sources conflict on the financial incentive amounts for Level 2 charging ports:

- **Per-port rebate:** Source A offers 2,000 *perport, while Source B offers* 1,500 per port (\$500 difference).
- **Maximum rebate cap:** Source A caps the total rebate at 20,000, *while Source B caps it at* 15,000 (\$5,000 difference). (Note: Both caps represent the exact equivalent of 10 ports at their respective per-port rates).

Evidence Assessment

Neither source provides external evidence, a publication date, or a version number to prove its current validity. Therefore, neither source can be chosen over the other without further verification.

Verification Steps Before Customer Presentation

Before presenting this incentive to a customer, you must verify the following with the official program administrator or governing website:

1. **Current Program Guidelines:** Check the most up-to-date, official program documentation or terms and conditions to confirm the exact active rebate structure.
2. **Effective Dates:** Determine if Source A represents an older/expired rate or a newly updated/future rate, and vice versa for Source B.
3. **Eligibility and Tiers:** Confirm if the different rates apply to specific customer sub-classes (e.g., general commercial vs. disadvantaged communities or fleet-specific caps).

Experiment 5 Evaluation

The model handled the conflicting information well. It identified both differences between the sources: the rebate amount per port and the maximum total rebate. More importantly, it did not choose one source over the other without evidence.

It recognized that neither source included enough information to determine which one was current, so it recommended checking the official program documentation, effective dates, or program administrator before using the incentive with a customer. That is the behavior I want from the final assistant.

The response also suggested a few possible explanations for the difference, such as different customer tiers or program versions. Those details were not included in the source information, but the model presented them as possibilities to verify rather than confirmed facts.

Overall, this was the strongest of the five experiments because the model recognized uncertainty and did not present conflicting information as settled.

Overall Prompt Evaluation

The five experiments show that the model can handle many of the tasks needed for the proposed incentive assistant. It was generally strong at explaining program requirements, organizing complicated conditions, separating assumptions from known information, and identifying conflicting sources.

The most consistent weakness was knowing when to stop. In Experiment 1, the model recognized that the utility provider was missing but still continued with an estimate. In Experiment 4, it recognized that the number of chargers did not necessarily equal the number of charging ports but still made an assumption and moved forward. Experiment 3 also showed that the model can subtly strengthen source language or calculate from an unconfirmed assumption.

Those results don't mean the project is a poor fit for generative AI, but it does show where the behavioral guardrails need to be stronger.

One of the most useful lessons from these tests is that the assistant needs a clearer definition of what counts as **confirmed**. If a required input has not met that definition, the model should ask for clarification instead of continuing to problem-solve around the missing information.

Fine-Tuning Approach

Fine-tuning should focus on the assistant's behavior rather than on current incentive facts.

Program amounts, deadlines, funding status, and eligibility rules can change, so storing those changing facts in a fine-tuned model would make the system harder to keep current. Retrieval is better suited for current program information.

Fine-tuning would instead use examples to reinforce the behaviors that the prompt experiments showed still need work. A training example could contain a customer scenario, retrieved program evidence, and an ideal response. Across many examples, the model could learn patterns such as stopping when a required input is missing, preserving the exact meaning of source language, separating confirmed facts from

assumptions, asking focused follow-up questions, and refusing unsupported savings estimates.

The experiments make those priorities concrete. Experiments 1 and 4 show that the model can notice a missing or ambiguous input and still continue. Experiment 3 shows that it can strengthen source language or move too quickly from an assumption to a calculation.

Prompt engineering and in-context examples should still be tested before committing to fine-tuning. If stronger prompts solve the problem consistently, fine-tuning may not be necessary. If the same behaviors keep appearing across many scenarios, fine-tuning becomes more useful because it can reinforce those response patterns across the application.

Next Steps

The next step is to strengthen the prompts around missing information and unsupported assumptions. The model needs clearer rules for when it should stop, ask a follow-up question, and wait for confirmation before continuing.

The customer intake should also be progressive. The assistant should begin with the information already available and request additional details, such as project cost or charging-port count, only when a relevant incentive requires them.

After those behaviors are more reliable, later development can add live incentive data, utility matching, source verification, and a more scalable approach to incentive calculations.

Conclusion

The Milestone 2 experiments support the potential of a Commercial Fleet EV Incentive Assistant. The model was generally effective at explaining program requirements, organizing complicated information, and identifying conflicts between sources.

The biggest weakness was knowing when to stop. In several experiments, the model recognized that information was missing or uncertain but still continued with an assumption or estimate. That makes stronger stop conditions and clearer definitions of confirmed information important before the assistant is used with live customer data.

At this point, the project still appears very feasible, but the language model should be treated as an interpretation and communication layer rather than the final authority on eligibility or financial calculations. Combining current program data, controlled model behavior, transparent calculations, and human verification remains the most practical direction for the project.

References

N.C. Clean Energy Technology Center. (n.d.). *DSIRE API*. <https://dsireusa.org/dsire-api/>

U.S. Census Bureau. (n.d.). *Census Geocoder documentation*.
<https://www.census.gov/programs-surveys/geography/technical-documentation/complete-technical-documentation/census-geocoder.html>

U.S. Department of Energy, Alternative Fuels Data Center. (n.d.-a). *About the laws and incentives data*. https://afdc.energy.gov/laws/data_methodology

U.S. Department of Energy, Alternative Fuels Data Center. (n.d.-b). *Data included in the laws and incentives download*.
https://afdc.energy.gov/data_download/laws_and_incentives_format

U.S. Department of Energy, Alternative Fuels Data Center. (n.d.-c). *Utility incentives*.
<https://afdc.energy.gov/laws/utility-incentives>

U.S. Energy Information Administration. (n.d.). *Annual Electric Power Industry Report, Form EIA-861 detailed data files*. <https://www.eia.gov/electricity/data/eia861/>